

All 144 scored episodes in broadcast order, shaded by combined score (59.3 to 94.3). Gaps separate shows.

■ lower ■ mid ■ higher □ scored from fewer than 3 sources

Technical prototype

TV Show Combined Ratings Aggregator

A runnable Python and SQL prototype. Synthetic episode ratings from three review sources, each on a different scale, combined into one trustworthy score per episode and per season.

444

SCORE SUBMISSIONS

144

EPISODES SCORED

193

QUALITY ISSUES CAUGHT

2

ENGINES: PANDAS + SQL

- IMDB rates 0-10, Rotten Tomatoes and Metacritic rate 0-100. Every score is converted onto one common scale before anything is averaged, or IMDB would be badly under-weighted
- A missing or bad source does not sink the whole episode. The combined score uses whichever sources are valid and says plainly how many of the three it rests on
- Show titles resolve against a reference crosswalk of canonical IDs to known aliases, built after catching a real bug where inconsistent spelling was silently splitting one episode into several

WHAT THE VALIDATION CAUGHT

193 exceptions across 444 submissions. One rule accounts for **148 of them** - the title-consistency check that surfaced the splitting bug, and the reason the crosswalk exists at all.

INCONSISTENT_SHOW_TITLE		148
DUPLICATE_SUBMISSION		24
MISSING_OR_INVALID_SCORE		11
SCORE_OUT_OF_RANGE		10

SAME LOGIC, TWO ENGINES

Python / pandas

```
# IMDB is 0-10, the other two 0-100
def normalize(row):
    if row["source"] == "imdb":
        return row["raw_score_num"] * 10
    return row["raw_score_num"]

clean["normalized_score"] = \
    clean.apply(normalize, axis=1)

episode_scores = (
    clean.groupby(["episode_id",
                  "canonical_title",
                  "season", "episode"])
    .agg(combined_score=(
        "normalized_score", "mean"),
        sources_used=("source",
                     "nunique")))
```

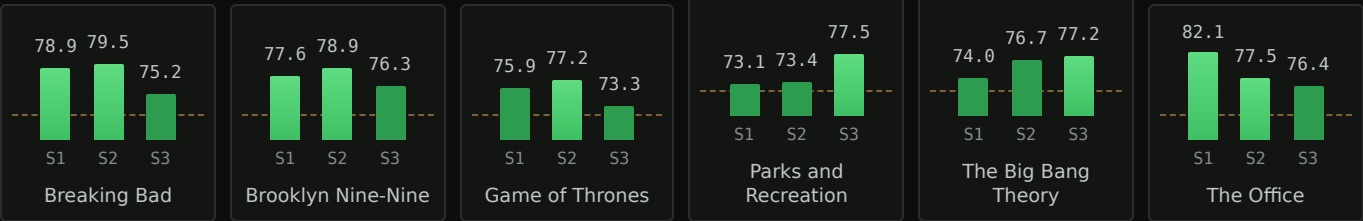
Standalone SQL

```
-- from a view that already normalized
-- scales and resolved canonical titles
SELECT episode_id,
       canonical_title,
       season,
       episode,
       ROUND(AVG(normalized_score), 1)
         AS combined_score,
       COUNT(*) AS sources_used
FROM view_normalized
GROUP BY episode_id,
       canonical_title,
       season, episode
ORDER BY episode_id;
```

Both engines produce identical combined scores on all 144 episodes. Writing the logic twice in two languages is the check - if they disagree, one of them is wrong.

SEASON SCORES

The generator builds 3 synthetic seasons of 8 episodes for each of 6 shows - 18 seasons, 144 episodes. Season scores range 73.1 to 82.1; the dashed line marks the 76.7 average.



RUN OUTPUT

pandas engine

```
Total score submissions: 444
Records with an exception: 176
Total exceptions raised: 193

Episodes with a combined score: 144
...based on fewer than 3 sources: 19
Seasons scored: 18
```

SQL engine, cross-checked

```
B99-S1E1 combined=74.7 sources=3
B99-S1E2 combined=65.3 sources=3
B99-S1E3 combined=75.3 sources=3
B99-S1E4 combined=87.5 sources=2

Breaking Bad S1 78.9 8 eps
Brooklyn 99 S1 77.6 8 eps
```

FILES

`generate_ratings.py` synthetic multi-source score generator
`show_reference.csv` title aliases to canonical ID
`validate_ratings.py` pandas quality checks and aggregation

`load_to_sqlite.py` loads the data into local SQLite
`reconcile.sql` standalone SQL checks and aggregation
`run_sql_reconciliation.py` sets up views, runs, prints
`README.md` design notes, the bug found and fixed, how to run

In a production system this same pattern - validate, normalize onto a common scale, combine, and stay transparent about data coverage - is how I handle data quality and reconciliation work in my current role, against fund and reference data rather than episode ratings.